Which Bits Can We Drop?

Component-Wise Quantization, Mixed-Precision Selection and QAT for a Learned Image Codec

Harshith (and team) | CS6886 Final Project | November 2026

Code: `https://github.com/harshith0518/CS6886-PULSE-Quantization`

Abstract. A learned image codec stores one weight file that must serve eight quality settings, so the usual recipe—pick one precision for the whole model—ignores that the model is wildly unbalanced. We measure how much each of nineteen components tolerates seven formats across eight quality settings and two datasets (69,233 measurements), and feed those costs to a mixed-integer program that chooses one precision per component. Result: **21.677 MB** against 54.688 MB FP32, a **60.4%** reduction at a **0.94%** worst-case rate-equivalent penalty and $+0.48$ to $+0.55\%$ BD-rate on three datasets. Against uniform INT8—which yields a *smaller* file at 26.18% penalty—the mixed assignment is **39**$\times$ better quality at comparable size. We then test quantization-aware fine-tuning (QAT): it removes **61–98%** of the damage on hand-chosen assignments and **exactly 0%** on the solver-optimised one, so QAT repairs what an assignment leaves on the table, and an optimal assignment leaves nothing.

1 Introduction

Post-training quantization is the standard way to shrink a neural codec, and the standard practice is a single precision for the whole model because per-layer analysis costs time. For *pulse-mse* [1] that reflex is expensive: 85.6% of its 14.18M parameters sit in six encoder blocks, while the entropy model—which dominates decode latency—holds 9.4%, and the 708k-parameter decoder is only 5.0% of the network yet contains a 4.7k block that loses 8% at 8 bits. Sensitivity is therefore not monotone in size: a 2.89M-parameter encoder block tolerates 8-bit weights at 0.5% penalty while that 4.7k renderer MLP loses 8%.

We therefore ask three questions. **(Q1)** Which components tolerate which precision, at which quality setting? **(Q2)** Given those measurements, what is the smallest per-component assignment that stays inside a quality budget? **(Q3)** Can QAT recover the residual loss, and where does it stop paying? A fourth, unplanned question proved the most useful: why does quantization not make this codec faster?

2 Prior work

Integer quantization with per-output-channel symmetric scales and straight-through gradient estimation is established practice [2, 3]. Learned image compression is represented here by PULSE [1]. Mixed-precision allocation as an integer program is a standard formulation [6]; we solve it with HiGHS through SciPy. For rate-distortion comparison we follow Bjøntegaard delta-rate [4], deviating only in using monotone piecewise-cubic interpolation [5] because we have eight points per curve rather than the four the original method assumes.

3 Idea and implementation

3.1 Measuring sensitivity

We quantize *one* component at a time, leaving every other component and all activations in FP32, and run the full codec over every image at every quality setting. Seven formats, nineteen components, two datasets (Kodak [7] screening, CLIC Professional [8] confirmation) and eight settings give 69,233 rows with zero failures and weight restoration verified by hash. An FP32 gate reproduces the reference bitstream and reconstruction hashes exactly at all eight settings, so any difference is attributable to the quantized component.

Quality is summarised by a rate-equivalent penalty that folds a quality change and a bitrate change into one number,

$$\text{penalty} = 100\left[\left(1+\tfrac{\Delta\text{bpp}}{100}\right)2^{-\Delta\text{PSNR}/\sigma}-1\right], \qquad (1)$$

where σ is the local slope of the reference curve. Zero means indistinguishable from FP32. The optimiser constrains the worst setting: the maximum over the eight settings of the mean over images.

Optimising over the sum of single-component costs assumes additivity, so we tested it: a uniform INT8 model's measured penalty against the sum of its nineteen component penalties gives a ratio of 0.96–1.05 on Kodak and 1.02–1.11 on CLIC. Additivity holds. A trap worth recording: summing each component's *own* worst setting inflates the total to 65.3% where the true value at a single setting is 26.7%; costs must be summed per setting, then maximised.

3.2 The allocation problem

Because one weight file serves all eight settings, a per-setting choice would not be deployable. The model has one binary variable per (component, format) pair, shared across settings, with one quality constraint per setting:

$$\begin{aligned}\min\ & \textstyle\sum_{c,f}\text{size}(c,f)\,x_{c,f}\\ \text{s.t.}\ & \textstyle\sum_{c,f}\text{pen}(c,f,q)\,x_{c,f}\le\epsilon \quad \forall q\in\{0,\dots,7\}\\ & \textstyle\sum_{f}x_{c,f}=1 \quad \forall c.\end{aligned}$$

The objective is bytes; deliberately not latency, because no per-format latency table exists for this GPU and an objective using one would optimise invented data. A greedy baseline is solved alongside.

3.3 Quantization-aware fine-tuning

QAT here means straight-through quantized convolutions followed by minimising distortion subject to holding the bitrate at its pre-quantization value. A fixed rate penalty fails in both directions—too weak and the model drifts down the RD curve, too strong and it destroys quality to hit the target—so we use an augmented Lagrangian whose multiplier is updated from the measured rate error on a fixed validation set:

$$\mathcal{L}=\frac{D}{D_{\text{ref}}}+\mu\left(\frac{R}{R_{\text{ref}}}-1\right), \qquad \mu\leftarrow\text{clip}(\mu+\alpha\,e_{\text{rate}}).$$

Entropy components are *frozen*: the shipped bundle holds precomputed integer CDF tables that would no longer describe the weights, breaking the encoder/decoder contract. The saved checkpoint must beat its own starting point, so a bad run cannot ship a worse model. Training uses 100 DIV2K [9] images disjoint from every test set, 12,000 steps at 512-pixel crops, about 100 minutes per tier on one 4 GB laptop GPU (PyTorch 2.6,

Table 1: Measured operating points. Worst-setting penalty (%), lower is better. FP32 is 54.688 MB; BD-rate for the champion is $+0.48/+0.55/+0.49\%$.

Point	MB	Save	Kodak	CLIC
1% champion	**21.677**	-60.4%	**0.67**	**0.85**
1.8% variant	18.664	-65.9%	1.27	1.60
INT8 tier + QAT	**14.401**	-73.7%	1.85	2.87
uniform INT8	13.904	-74.6%	26.18	56.70

Table 2: QAT at 12,000 steps. Worst-setting penalty (%) before and after; the last row used 8,000 steps.

Tier	MB	Before	After	Removed
INT8, 16 comps	14.401	8.61	**3.36**	61.0%
INT4, 2 blocks	13.572	119.18	**20.53**	82.8%
INT4, 4 blocks	11.369	1771.71	**41.84**	97.6%
ILP 2% assignment	18.389	1.936	1.936	**0.0%**

CUDA 12.4, HiGHS via SciPy)—2.2% of the optimiser steps and 0.2% of the sample budget of pretraining [1]. This is a repair operation, not retraining.

4 Evaluation

4.1 Against the baseline

Table 1 reports all three operating points, measured on all three test sets at all eight settings against the same FP32 reference. The solver predicted 0.9715% for the champion and measured 0.94%: the analytic model errs conservative on every dataset. Set14 was never used for fitting and holds, which is the best evidence that the assignment transfers.

The decisive comparison is the last row. Uniform INT8 produces a *smaller* file than our champion and is 39× worse on Kodak and 67× worse on CLIC. Uniform settings also vary 2.9× across datasets while mixed assignments vary at most 1.4×: constraining the total rather than each component buys robustness as well as quality.

4.2 What QAT contributed

Table 2 isolates the third question. QAT removes 61–98% of the damage on assignments chosen by hand, and *nothing at all* on the assignment the solver had already optimised. The pattern is the useful result: fine-tuning is a *substitute* for allocation, not a supplement. The size of the repair is set by what the assignment left on the table, and a mixed-integer program that has minimised total damage leaves almost nothing.

4.3 Why there is no speedup

No configuration measured faster. Paired interleaved benchmarking across 50 pairs per cell gives $+4.79\%$ latency for the mixed model (95% CI $+1.27$ to $+9.80$) and -0.39% energy, whose interval spans zero. The reason is arithmetic: a decode is 88 ms, of which 80 ms is entropy coding and 8 ms is the convolutional network. Quantization accelerates convolutions, and convolutions are 9% of a decode and 27% of a round trip. Simulated formats (FP8, INT4, FP4) carry no speed claim at all; only FP16 and BF16 run real kernels here, and they save energy rather than time.

5 Conclusion

Granularity, not precision, is the lever: mixed precision is worth 39× over uniform precision at comparable size. And QAT is a substitute for good allocation—it recovered 61–98% of the damage on hand-chosen assignments and 0% on an optimal one. Both halves matter, and the second is only visible because we measured the control case.

Limitations

(a) Our checkpoint selector under-reports QAT: a 6-crop validation probe scored the best INT8 model as no better than its start (-0.08%) while the full evaluation shows 8.61%→3.36%, so saved checkpoints are probably not the best available. (b) Weights only; activations remain FP32, so W8A8 is not exercised. (c) 4-bit is simulated, so its speed is unmodelled. (d) One entropy lane remains—the largest unused latency lever. (e) QAT results are one seed on 100 images; the qualitative finding holds across three tiers and two step counts, the exact percentages are single-run.

References

[1] Jia et al. *PULSE: a learned image codec*. arXiv:2609.18602, 2026.

[2] Jacob et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *CVPR*, 2018. arXiv:1712.05877.

[3] Bengio, Léonard, Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. arXiv:1308.3432, 2013.

[4] G. Bjøntegaard. Calculation of average PSNR differences between RD-curves. *ITU-T VCEG-M33*, 2001.

[5] Fritsch, Butland. A method for constructing local monotone piecewise cubic interpolants. *SIAM J. Sci. Comput.* 5(2):300–304, 1984. DOI 10.1137/0905021.

[6] Huangfu, Hall. Parallelizing the dual revised simplex method. *Math. Prog. Comp.* 10(1):119–142, 2018. DOI 10.1007/s12532-017-0130-5.

[7] Eastman Kodak Company. Kodak PhotoCD image dataset, 1991.

[8] Toderici et al. Workshop and challenge on learned image compression. *CVPR Workshops*, 2020.

[9] Agustsson, Timofte. NTIRE 2017 challenge on single image super-resolution. *CVPR Workshops*, 2017.

[10] Zeyde, Elad, Protter. On single image scale-up using sparse-representations. *Curves and Surfaces*, 2010.

[11] Our code and results: `https://github.com/harshith0518/CS6886-PULSE-Quantization`.